UAlg FCT
UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

Algoritmos e Estruturas de Dados

Exercícios

Capítulo 3. Coleções (listas, pilhas, filas)

3.1 Funções sobre listas

3.1.1 Implemente uma função interna para uma lista ligada, que a partir de um ponteiro para o primeiro nó, é capaz de reverter a lista de forma destrutiva.

3.1.2 Implemente uma função filtraLista, que recebe uma lista de objetos genéricos do tipo T, e um predicado (i.e. uma função que retorna *true* ou *false*) com um argumento genérico do tipo T, e devolve uma lista apenas com os elementos da lista que satisfazem o predicado.

3.1.3 Utilize o método anterior para implementar uma função filtaLista10, que dada uma lista de números inteiros devolve uma lista apenas com os números inferiores a 10.

3.1.4 Implemente uma função mapeia, que recebe uma lista de objetos genéricos do tipo T, e uma função *f*, e devolve uma nova lista que se obtém de aplicar a função f a todos os elementos da lista inicial.

3.1.5 Implemente uma função *any*, que dada uma lista de objetos genéricos do tipo T, e um predicado com um argumento genérico do tipo T, devolve *true* se algum dos elementos da lista satisfaz o predicado, e false caso contrário.

3.1.6 Implemente uma função *accumulate*, que dada uma lista de objetos genéricos do tipo T, e um operador binário genérico do tipo T, e devolve o resultado total resultante de se aplicar o operador a todos os elementos da lista. Por exemplo, se o operador corresponder a uma soma, a função *accumulate* retorna a soma de todos os elementos da lista.

3.1.7 Utilize o método anterior para implementar a função piatório, que dada uma lista de inteiros, multiplica todos os elementos da lista.

3.2 Pilhas e Filas

3.2.1 Indique o que acontece à fila q com o seguinte código:

```
Stack<String> stack = new Stack<String>();
while(!q.isEmpty())
{
    stack.push(q.dequeue());
}
while(!stack.isEmpty())
{
    q.enqueue(stack.pop());
}
```

3.2.2 Suponha que um cliente executa uma sequência de operações *push* e *pop* sobre uma pilha (estas operações podem ser feitas por qualquer ordem). As operações de *push* colocam inteiros entre 0 e 9 na pilha pela ordem correspondente. As operações de *pop* imprimem o elemento retornado. Quais das seguintes sequências não podem acontecer?

a) 4 3 2 1 0 9 8 7 6 5
b) 4 6 8 7 5 3 2 9 0 1
c) 2 5 6 7 4 8 9 3 1 0
d) 4 3 2 1 0 5 6 7 8 9
e) 1 2 3 4 5 6 9 8 7 0
f) 0 4 6 5 3 8 1 7 2 9
g) 1 4 7 9 8 6 5 3 0 2
h) 2 1 4 3 6 5 8 7 9 0

3.2.3 Suponha que um cliente executa uma sequência de operações en*queue* e *dequeue* sobre uma fila (estas operações podem ser feitas por qualquer ordem). As operações de *enqueue* colocam inteiros entre 0 e 9 na pilha pela ordem correspondente. As operações de *dequeue* imprimem o elemento retornado. Quais das seguintes sequências não podem acontecer?

a) 0 1 2 3 4 5 6 7 8 9
b) 4 6 8 7 5 3 2 9 0 1
c) 2 5 6 7 4 8 9 3 1 0
d) 4 3 2 1 0 5 6 7 8 9

3.2.4 Escreva um método parenteses que dada uma *string* correspondente a uma linha de texto, verifica se os parênteses (retos, curvos ou chavetas) estão bem balanceados. Por exemplo, o seu método deverá retornar *true* para "[()]{}{[()()]()}" e *false* para "[(])".

3.2.5 Escreva um programa que recebe do *standard input* uma expressão aritmética sem parênteses esquerdos, e imprime no ecrã a expressão infixa equivalente com os parênteses inseridos. Por exemplo, para o *input*

1 + 2) * 3 – 4) * 5 – 6)))

o seu programa deverá imprimir

((1 + 2) * ((3 – 4) * (5 – 6)))